**NAME: ANUPAM DESHMUKH**
**CLASS:LY-28(21 PATTERN)**
**ROLL NO: 2223389**
**ENROLLMENT: MITU22BTCS0134**
**SUBJECT: HPCL LAB**

# Lab Assignment 2: Concurrent Programming

## Question 1: Race Conditions and Locks

### Conceptual Questions

**1. What is a race condition?**

A race condition occurs when two or more threads access shared data concurrently and at least one of them modifies it, and the final outcome depends on the unpredictable timing/order in which the threads are scheduled. The program's correctness becomes a matter of luck rather than logic.

**2. What is a critical section?**

A critical section is the segment of code that accesses shared resources (variables, files, data structures) which must not be executed by more than one thread at the same time, to prevent race conditions.

**3. What is the difference between shared and private data in threads?**

Shared data (e.g. global/static variables, heap memory) is visible to and modifiable by every thread in the process, so it needs synchronization. Private data (e.g. local stack variables) belongs to a single thread only — each thread has its own copy/stack — so it needs no synchronization.

**4. Why is counter++ not necessarily an atomic operation?**

At the machine level counter++ expands into at least three separate steps: (1) LOAD the current value of counter from memory into a register, (2) INCREMENT the register, (3) STORE the register back to memory. A thread can be preempted between any of these steps, so two threads can both load the same old value, both increment it locally, and both write back the same result, silently losing one increment (a 'lost update').

**5. What happens if a thread fails to release a lock?**

Every other thread waiting on that lock blocks forever, since the mutex never becomes available again. This causes a deadlock (or at minimum permanent starvation) for all threads waiting on that same mutex, and the program hangs.

**6. Differentiate between a mutex and a semaphore.**

A mutex is a binary lock with the concept of ownership: only the thread that locked it may unlock it, and it is used purely for mutual exclusion of a critical section. A semaphore is a counting synchronization primitive (no ownership) that can allow more than one thread through at a time (based on its count) and is used both for mutual exclusion (binary semaphore) and for signaling/coordination between threads (e.g. producer-consumer).

**7. Does using a lock make a program parallel? Justify.**

No. A lock does the opposite inside its protected region — it serializes access to the critical section so that only one thread executes it at a time. The program can still be parallel outside the critical section (threads run concurrently on multiple cores), but the locked region itself is never executed in parallel; it becomes a strictly sequential bottleneck.

**8. How does a lock affect CPU utilization when many threads compete for it?**

As contention increases, more threads spend time waiting instead of doing useful work. With blocking mutexes the waiting threads are put to sleep (context-switched out), which wastes CPU cycles on context-switch overhead rather than computation. With spinlocks, waiting threads busy-wait, consuming 100% CPU without making progress. Either way, heavy contention lowers effective CPU utilization and throughput even though the CPU may appear 'busy'.

## Programming Assignment

### Part A: Without Synchronization (source: q1_nolock.c)

Four threads each increment a shared long counter 100,000 times with a plain counter++ and no lock.

```
// Q1 Part A: Shared counter incremented by 4 threads WITHOUT synchronization
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

#define NUM_THREADS 4
#define INCREMENTS 100000

long counter = 0;

void *increment_no_lock(void *arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        counter++;              // NOT atomic: read-modify-write race
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    struct timespec start, end;

    counter = 0;
    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, increment_no_lock, NULL);
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);
    double elapsed = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("[No Lock] Final counter value = %ld (expected 400000)\n", counter);
    printf("[No Lock] Time taken = %.6f seconds\n", elapsed);
    return 0;
}
```

Part B: Using a Mutex (source: q1_lock.c)

Same setup, but every increment is wrapped in pthread_mutex_lock/unlock.

```
// Q1 Part B: Shared counter incremented by 4 threads WITH a mutex
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

#define NUM_THREADS 4
#define INCREMENTS 100000

long counter = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *increment_with_lock(void *arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        pthread_mutex_lock(&mutex);   // enter critical section
        counter++;
        pthread_mutex_unlock(&mutex); // leave critical section
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    struct timespec start, end;

    counter = 0;
    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, increment_with_lock, NULL);
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);
    double elapsed = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("[Mutex]   Final counter value = %ld (expected 400000)\n", counter);
    printf("[Mutex]   Time taken = %.6f seconds\n", elapsed);
    pthread_mutex_destroy(&mutex);
    return 0;
}
```

Output — 5 runs of each version (actual captured output)

```
=== NO LOCK: 5 runs ===
[No Lock] Final counter value = 400000 (expected 400000)
[No Lock] Time taken = 0.002049 seconds
[No Lock] Final counter value = 400000 (expected 400000)
[No Lock] Time taken = 0.001907 seconds
[No Lock] Final counter value = 400000 (expected 400000)
[No Lock] Time taken = 0.002021 seconds
[No Lock] Final counter value = 400000 (expected 400000)
[No Lock] Time taken = 0.001991 seconds
[No Lock] Final counter value = 400000 (expected 400000)
[No Lock] Time taken = 0.001314 seconds
```

```
=== MUTEX: 5 runs ===
[Mutex]   Final counter value = 400000 (expected 400000)
[Mutex]   Time taken = 0.007047 seconds
[Mutex]   Final counter value = 400000 (expected 400000)
[Mutex]   Time taken = 0.007171 seconds
[Mutex]   Final counter value = 400000 (expected 400000)
[Mutex]   Time taken = 0.006874 seconds
[Mutex]   Final counter value = 400000 (expected 400000)
[Mutex]   Time taken = 0.007038 seconds
[Mutex]   Final counter value = 400000 (expected 400000)
[Mutex]   Time taken = 0.006934 seconds

=== SEQUENTIAL (baseline): 1 run ===
[Sequential] Final counter value = 400000 (expected 400000)
[Sequential] Time taken = 0.000281 seconds
```

**Important, honest observation about the No-Lock results:**

On the container this was executed in, the machine exposes a single CPU core (nproc = 1). With only one core, the Linux scheduler happens to let each 100,000-iteration thread run to completion within a single time slice, so the interleaving that causes lost updates never actually occurs here — all 5 runs of the unsynchronized version still landed on exactly 400000. This is a real, reproducible property of this environment, not a claim that counter++ is safe. On a normal multi-core machine (which is what this assignment expects you to run on), the four threads execute truly simultaneously, and you should expect the No-Lock final value to be LESS than 400000 and to vary between runs, because of the lost-update race on the load-increment-store sequence.

To make the race mechanism directly observable regardless of core count, a third variant, q1_nolock_yield.c, inserts a sched_yield() between the load and the store of each increment to force a context switch mid-operation. Running that program (real captured output) reliably reproduces lost updates:

```
Final counter = 100001 (expected 400000)
Final counter = 100001 (expected 400000)
Final counter = 100001 (expected 400000)
```

This confirms the race condition concretely: with forced interleaving, three of the four threads' increments are almost entirely lost because they all read the same stale value before any of them writes back.

## Execution-Time Comparison

| Version | Final Counter Value | Typical Time (s) | Correct? |
|---|---|---|---|
| Sequential (1 thread, baseline) | 400000 | ~0.00028 | Yes |
| Multithreaded, no mutex | 400000 (this 1-core sandbox) / may be < 400000 on multi-core hardware | ~0.0018 | Unreliable in general |
| Multithreaded, with mutex | 400000 (every run) | ~0.0070 | Yes, always |

The mutex version is about 3.5× slower than the unsynchronized version here, even though both do the same 400,000 increments. Every single increment must acquire and release the mutex, and pthread_mutex_lock/unlock involve an atomic compare-and-swap plus, under contention, a kernel futex call — so the lock overhead dominates when the critical section (a single ++) is this tiny. The sequential version is fastest of all because it has zero thread-creation, scheduling, or locking overhead.

## Why the Results Differ

Without a lock, counter++ is three unprotected machine steps (load, add, store). When multiple threads interleave these steps, updates can be silently overwritten — e.g. two threads both load the value 57, both compute 58, and both store 58, so one increment vanishes. The more times this interleaving happens, the further the final result drifts below 400,000, and the exact amount differs run to run because scheduling is non-deterministic. With the mutex, only one thread can be inside the load-add-store sequence at any instant, so every increment is guaranteed to be preserved, and the result is deterministically 400,000 every time — at the cost of serializing that section and adding lock/unlock overhead.

## Critical Section Identification

The critical section in this program is the single line:

```
counter++;
```

This is the only line that reads and writes the shared variable counter. In Part B it is exactly the code wrapped between pthread_mutex_lock(&mutex) and pthread_mutex_unlock(&mutex).

# Question 2: Bounded Producer-Consumer Problem

## Conceptual Questions

**1. What do the producer and consumer represent in a computing system?**

The producer represents any entity that generates data or work faster than it can be described in a single step — a sensor, a network receiver, a data-generation thread, a web-server request handler. The consumer represents the entity that processes that data — a worker thread, a database writer, a rendering pipeline, or a CPU/GPU compute stage.

**2. Why should the buffer have a fixed capacity?**

A fixed (bounded) capacity limits the amount of memory used to hold unconsumed items and creates natural backpressure: if the buffer fills, producers must pause, preventing runaway/unbounded memory growth when the consumer temporarily falls behind. An unbounded buffer risks exhausting memory if the producer is persistently faster than the consumer.

**3. What happens when the producer is faster than the consumer?**

Items accumulate in the buffer until it becomes full. Once full, the producer must block (wait on the empty semaphore) until the consumer frees a slot, which throttles the producer down to the consumer's effective rate. This is exactly the backpressure mechanism a bounded buffer is meant to provide.

**4. What happens when the consumer is faster than the producer?**

The buffer stays close to empty (or drains to empty) and the consumer repeatedly blocks on the full semaphore waiting for new items, spending most of its time idle/sleeping rather than doing wasted work.

**5. Why is a mutex required even when empty and full semaphores are used?**

The empty/full semaphores only control HOW MANY items/slots exist; they do not protect the actual buffer indices (in, out) or the shared item array from being corrupted when more than one producer (or more than one consumer) modifies them concurrently. With 2 producers and 2 consumers here, the mutex is essential to serialize the actual read/write of the shared in/out indices and array, which the counting semaphores alone cannot do.

**6. What is buffer overflow?**

An attempt to insert an item into the buffer when it is already at full capacity, i.e. more items are pushed than there are free slots. In this design it is prevented entirely because sem_wait(&empty) blocks the producer until a slot is genuinely free.

**7. What is buffer underflow?**

An attempt to remove an item from the buffer when it is empty, i.e. more items are requested than are available. It is prevented here because sem_wait(&full) blocks the consumer until an item genuinely exists.

**8. Explain backpressure using the Producer-Consumer problem.**

Backpressure is the mechanism by which a slow consumer's limited processing rate is communicated back up the pipeline to slow down the producer, instead of letting unconsumed work pile up without bound. In this implementation, backpressure is implemented directly by the empty semaphore: when the buffer is full, sem_wait(&empty) blocks every producer, so the producers cannot outpace what the consumers can actually drain.

## Programming Assignment (source: q2_prodcons.c)

2 producers, 2 consumers, bounded circular buffer of capacity 5, 10 items per producer (20 items total). Termination uses the classic poison-pill pattern: once both producers finish, one sentinel value is pushed per consumer so that every consumer thread wakes and exits cleanly with no risk of deadlock.

```
// Q2: Bounded Producer-Consumer using mutex + counting semaphores (empty/full)
// Configurable producer/consumer delay via command-line args so we can run
// the three required experiments (equal speed / producer-fast / consumer-fast).
// Termination uses the classic "poison pill" pattern: once producers finish,
// one sentinel value (-1) is pushed into the buffer for each consumer thread,
// so no consumer can ever block forever waiting on an item that never comes.
//
// Usage: ./q2_prodcons <producer_delay_ms> <consumer_delay_ms>

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 5
#define NUM_PRODUCERS 2
#define NUM_CONSUMERS 2
#define ITEMS_PER_PRODUCER 10
#define SENTINEL (-1)

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
int buffer_count = 0;

pthread_mutex_t mutex;
sem_t empty;   // counts free slots
sem_t full;    // counts filled slots

int producer_delay_ms = 50;
int consumer_delay_ms = 50;

int full_events = 0;   // number of times buffer hit capacity
int empty_events = 0;  // number of times buffer hit zero

void msleep(int ms) {
    struct timespec ts;
    ts.tv_sec = ms / 1000;
    ts.tv_nsec = (ms % 1000) * 1000000L;
    nanosleep(&ts, NULL);
}

void push_item(int item, int announce, long producer_id) {
    sem_wait(&empty);
    pthread_mutex_lock(&mutex);

    buffer[in] = item;
    in = (in + 1) % BUFFER_SIZE;
    buffer_count++;
    if (announce) {
        printf("Producer %ld produced %d | Buffer count: %d\n", producer_id, item,
buffer_count);
        if (buffer_count == BUFFER_SIZE) {
            full_events++;
            printf("   >> Buffer FULL (capacity %d)\n", BUFFER_SIZE);
        }
    }
```

```
    pthread_mutex_unlock(&mutex);
    sem_post(&full);
}

void *producer(void *arg) {
    long id = (long)arg;
    unsigned int seed = (unsigned int)time(NULL) ^ (unsigned int)(id * 7919);

    for (int i = 0; i < ITEMS_PER_PRODUCER; i++) {
        int item = rand_r(&seed) % 100;
        msleep(producer_delay_ms);          // simulate generation time
        push_item(item, 1, id);
    }
    return NULL;
}

void *consumer(void *arg) {
    long id = (long)arg;

    while (1) {
        sem_wait(&full);                     // wait for an available item
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        buffer_count--;

        if (item == SENTINEL) {
            pthread_mutex_unlock(&mutex);
            sem_post(&empty);
            break;                           // poison pill -> this consumer exits
        }

        printf("Consumer %ld consumed %d | Buffer count: %d\n", id, item, buffer_count);
        if (buffer_count == 0) {
            empty_events++;
            printf("   >> Buffer EMPTY\n");
        }

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);                    // signal a free slot

        msleep(consumer_delay_ms);           // simulate processing time
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc >= 3) {
        producer_delay_ms = atoi(argv[1]);
        consumer_delay_ms = atoi(argv[2]);
    }

    pthread_t producers[NUM_PRODUCERS], consumers[NUM_CONSUMERS];
    pthread_mutex_init(&mutex, NULL);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    printf("=== Experiment: producer_delay=%dms consumer_delay=%dms ===\n",
           producer_delay_ms, consumer_delay_ms);
```

```
    for (long i = 0; i < NUM_PRODUCERS; i++)
        pthread_create(&producers[i], NULL, producer, (void *)(i + 1));
    for (long i = 0; i < NUM_CONSUMERS; i++)
        pthread_create(&consumers[i], NULL, consumer, (void *)(i + 1));

    for (int i = 0; i < NUM_PRODUCERS; i++) pthread_join(producers[i], NULL);

    // All real items have been produced. Push one sentinel per consumer so
    // each consumer thread wakes up, sees it, and terminates cleanly.
    for (int i = 0; i < NUM_CONSUMERS; i++)
        push_item(SENTINEL, 0, 0);

    for (int i = 0; i < NUM_CONSUMERS; i++) pthread_join(consumers[i], NULL);

    printf("=== Summary: buffer became FULL %d time(s), became EMPTY %d time(s) ===\n",
           full_events, empty_events);

    pthread_mutex_destroy(&mutex);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

Experiment 1: Producer and Consumer at Approximately Equal Speed (50ms / 50ms)

```
=== Experiment: producer_delay=50ms consumer_delay=50ms ===
Producer 1 produced 44 | Buffer count: 1
Consumer 1 consumed 44 | Buffer count: 0
   >> Buffer EMPTY
Producer 2 produced 13 | Buffer count: 1
Consumer 2 consumed 13 | Buffer count: 0
   >> Buffer EMPTY
Producer 1 produced 70 | Buffer count: 1
Consumer 1 consumed 70 | Buffer count: 0
   >> Buffer EMPTY
...
Producer 2 produced 51 | Buffer count: 1
Producer 1 produced 73 | Buffer count: 2
Consumer 2 consumed 51 | Buffer count: 1
Consumer 1 consumed 73 | Buffer count: 2
=== Summary: buffer became FULL 0 time(s), became EMPTY 18 time(s) ===
```

At equal speeds the buffer oscillates between 0 and 1-2 items and never approaches capacity 5; it hit EMPTY 18 times and FULL 0 times.

Experiment 2: Producer Faster Than Consumer (10ms / 80ms)

```
=== Experiment: producer_delay=10ms consumer_delay=80ms ===
Producer 1 produced 52 | Buffer count: 1
Producer 2 produced 89 | Buffer count: 2
Consumer 1 consumed 52 | Buffer count: 1
Consumer 2 consumed 89 | Buffer count: 0
   >> Buffer EMPTY
Producer 2 produced 25 | Buffer count: 1
Producer 1 produced 75 | Buffer count: 2
Producer 2 produced 53 | Buffer count: 3
Producer 1 produced 77 | Buffer count: 4
Producer 1 produced 43 | Buffer count: 5
   >> Buffer FULL (capacity 5)
Consumer 2 consumed 25 | Buffer count: 4
...
```

```
Producer 2 produced 77 | Buffer count: 4
Consumer 2 consumed 93 | Buffer count: 4
Consumer 1 consumed 2 | Buffer count: 4
Consumer 2 consumed 42 | Buffer count: 3
Consumer 1 consumed 77 | Buffer count: 2
=== Summary: buffer became FULL 9 time(s), became EMPTY 1 time(s) ===
```

With a fast producer and a slow consumer, the buffer fills up repeatedly (FULL 9 times) and producers visibly stall on sem_wait(&empty) until the consumer frees a slot — exactly the expected backpressure behaviour.

Experiment 3: Consumer Faster Than Producer (80ms / 10ms)

```
=== Experiment: producer_delay=80ms consumer_delay=10ms ===
Producer 1 produced 10 | Buffer count: 1
Consumer 1 consumed 10 | Buffer count: 0
   >> Buffer EMPTY
Producer 2 produced 35 | Buffer count: 1
Consumer 2 consumed 35 | Buffer count: 0
   >> Buffer EMPTY
Producer 1 produced 90 | Buffer count: 1
Producer 2 produced 96 | Buffer count: 2
Consumer 2 consumed 90 | Buffer count: 1
Consumer 1 consumed 96 | Buffer count: 0
   >> Buffer EMPTY
...
Producer 2 produced 18 | Buffer count: 1
Producer 1 produced 79 | Buffer count: 2
Consumer 1 consumed 18 | Buffer count: 3
Consumer 2 consumed 79 | Buffer count: 2
=== Summary: buffer became FULL 0 time(s), became EMPTY 10 time(s) ===
```

With a slow producer and fast consumer, the buffer stays almost empty throughout (EMPTY 10 times, FULL 0 times); consumers repeatedly block on sem_wait(&full) waiting for new items.

Buffer-State Table

| Experiment | Producer Delay | Consumer Delay | Times FULL | Times EMPTY | Behaviour |
|---|---|---|---|---|---|
| 1: Equal speed | 50 ms | 50 ms | 0 | 18 | Oscillates near-empty, no backpressure needed |
| 2: Producer faster | 10 ms | 80 ms | 9 | 1 | Buffer saturates — producers repeatedly blocked |
| 3: Consumer faster | 80 ms | 10 ms | 0 | 10 | Buffer stays near-empty — consumers repeatedly blocked |

Explanation of mutex, empty, and full

empty (initialized to BUFFER_SIZE = 5): counts the number of free slots. A producer calls sem_wait(&empty) before inserting, guaranteeing it never overflows the buffer.

full (initialized to 0): counts the number of filled slots. A consumer calls sem_wait(&full) before removing, guaranteeing it never underflows an empty buffer.

mutex: protects the actual critical section — the shared in/out indices and buffer_count — from being corrupted when two producers (or two consumers) try to update them at the same instant. The semaphores control quantity/availability; the mutex controls exclusive access to the shared bookkeeping data.

## Real-World Application

A concrete real application is a web server's request-handling pipeline: network I/O threads (producers) place incoming HTTP requests into a fixed-size in-memory request queue, and a pool of worker threads (consumers) pull requests off that queue to process them. The bounded queue prevents an unbounded burst of client requests from exhausting server memory; when the queue fills, new connections experience backpressure (delay or a 503 response) until workers catch up — precisely the empty/full/mutex pattern implemented above.

# Question 3: Dining Philosophers and Deadlock Prevention

## Conceptual Questions

**1. What do philosophers and forks represent in computing?**

Philosophers represent concurrent processes or threads competing for resources. Forks represent shared, exclusively-lockable resources (e.g. mutexes, files, database rows, devices) that a process must acquire before it can proceed with its work ("eating").

**2. Why does every philosopher require two forks?**

Eating is modeled as requiring two simultaneously-held exclusive resources (the fork on each side), mirroring real systems where a task needs multiple locked resources at once (e.g. transferring money needs locks on two different bank accounts). Needing two resources simultaneously, shared with neighbors, is exactly what creates the potential for circular wait.

**3. Explain the four necessary conditions for deadlock.**

(1) Mutual exclusion: a resource can be held by only one process at a time. (2) Hold and wait: a process holds at least one resource while waiting to acquire additional resources held by others. (3) No preemption: a resource cannot be forcibly taken away from a process; it must be released voluntarily. (4) Circular wait: a closed chain of two or more processes exists where each is waiting for a resource held by the next process in the chain. All four must hold simultaneously for deadlock to occur; breaking any one prevents it.

**4. What is circular wait?**

A cycle of processes P1, P2, ..., Pn where P1 waits for a resource held by P2, P2 waits for a resource held by P3, ..., and Pn waits for a resource held by P1 — closing the loop so that no process in the cycle can ever proceed.

**5. Differentiate between deadlock and starvation.**

Deadlock is a permanent standstill: the involved processes can never make progress again without external intervention (e.g. killing a process). Starvation is a situation where a specific process is repeatedly denied a resource it needs (e.g. always losing the race for a fork to a neighbor) and makes no progress for a long or unbounded time, even though the overall system continues to make progress — other processes are still running fine.

**6. How can resource ordering prevent deadlock?**

By imposing a global, fixed order on all resources (e.g. numbering forks 0-4) and requiring every process to acquire resources strictly in increasing order, it becomes impossible to form a cycle: in any cycle there would have to be some process that acquires a lower-numbered resource after already holding a higher-numbered one, which is forbidden by the ordering rule. This directly breaks the circular-wait condition.

**7. Can a program be free from deadlock but still experience starvation?**

Yes. Resource ordering, for example, guarantees no deadlock, but an unlucky/unfair scheduler could still repeatedly favor certain philosophers over one particular neighbor, so that specific philosopher rarely gets both forks even though the system as a whole keeps progressing. Deadlock-freedom is about global progress; starvation-freedom is about fairness to each individual process.

**8. What is the role of a waiter or arbitrator?**

A waiter/arbitrator is a central coordinator (implemented here as a counting semaphore) that limits how many processes may simultaneously attempt to acquire the shared resources — e.g. allowing at most N-1 of N philosophers to try picking up forks at once. This guarantees that at least one philosopher can always complete its acquisition, which makes a full circular wait involving all N processes impossible.

## Part A: Deadlock-Prone Implementation (source: q3a_deadlock.c)

Every philosopher acquires the LEFT fork first, waits briefly, then tries the RIGHT fork — the classic setup for circular wait. A watchdog reports whether any philosopher managed to eat.

```
// Q3 Part A: Deadlock-prone Dining Philosophers.
// Every philosopher picks up LEFT fork first, then RIGHT fork -> classic
// circular-wait setup. Run with a watchdog timer: if no philosopher manages
// to eat within TIMEOUT seconds after everyone has picked up one fork, the
// program declares deadlock and exits (a truly deadlocked program would
// otherwise hang forever).

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

#define N 5
#define TIMEOUT_SECONDS 5

pthread_mutex_t forks[N];
int meals_eaten[N] = {0};
volatile int someone_ate = 0;
volatile int stop = 0;

void msleep(int ms) {
    struct timespec ts = {ms / 1000, (ms % 1000) * 1000000L};
    nanosleep(&ts, NULL);
}

void *philosopher(void *arg) {
    int id = (int)(long)arg;
    int left = id;
    int right = (id + 1) % N;

    while (!stop) {
        printf("Philosopher %d is thinking\n", id);
        msleep(100);

        printf("Philosopher %d is hungry, waiting for left fork %d\n", id, left);
        pthread_mutex_lock(&forks[left]);
        printf("Philosopher %d picked up left fork %d\n", id, left);

        msleep(50); // deliberate delay: gives every philosopher time to grab
                    // its left fork before anyone tries the right one,
                    // guaranteeing the circular-wait deadlock condition.

        printf("Philosopher %d is waiting for right fork %d\n", id, right);
        pthread_mutex_lock(&forks[right]);
        printf("Philosopher %d picked up right fork %d\n", id, right);

        printf("Philosopher %d is EATING\n", id);
        meals_eaten[id]++;
        someone_ate = 1;
        msleep(50);

        pthread_mutex_unlock(&forks[right]);
        pthread_mutex_unlock(&forks[left]);
        printf("Philosopher %d released both forks\n", id);
    }
    return NULL;
```

```
}

int main() {
    pthread_t th[N];
    for (int i = 0; i < N; i++) pthread_mutex_init(&forks[i], NULL);
    for (long i = 0; i < N; i++) pthread_create(&th[i], NULL, philosopher, (void *)i);

    // Watchdog: after everyone has had a chance to grab a left fork,
    // wait TIMEOUT_SECONDS; if nobody ate, we are deadlocked.
    sleep(1); // let philosophers reach the "waiting for right fork" state
    for (int s = 0; s < TIMEOUT_SECONDS; s++) {
        someone_ate = 0;
        sleep(1);
        if (someone_ate) {
            // progress is happening, program is fine, let it run a bit more
        }
    }

    stop = 1;
    printf("\n===== WATCHDOG REPORT (Part A: deadlock-prone) =====\n");
    printf("If no 'EATING' lines appear above after the initial rush, the\n");
    printf("system is deadlocked: every philosopher holds its left fork and\n");
    printf("waits forever for its right fork (circular wait).\n");
    for (int i = 0; i < N; i++)
        printf("Philosopher %d ate %d time(s)\n", i, meals_eaten[i]);

    // NOTE: threads that are stuck on pthread_mutex_lock cannot be joined;
    // we exit the process directly to end the demonstration.
    exit(0);
}
```

Actual Output (Part A) — Real Deadlock Reproduced

```
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 4 is hungry, waiting for left fork 4
Philosopher 4 picked up left fork 4
Philosopher 3 is hungry, waiting for left fork 3
Philosopher 3 picked up left fork 3
Philosopher 2 is hungry, waiting for left fork 2
Philosopher 2 picked up left fork 2
Philosopher 1 is hungry, waiting for left fork 1
Philosopher 1 picked up left fork 1
Philosopher 0 is hungry, waiting for left fork 0
Philosopher 0 picked up left fork 0
Philosopher 1 is waiting for right fork 2
Philosopher 2 is waiting for right fork 3
Philosopher 3 is waiting for right fork 4
Philosopher 0 is waiting for right fork 1
Philosopher 4 is waiting for right fork 0

===== WATCHDOG REPORT (Part A: deadlock-prone) =====
If no 'EATING' lines appear above after the initial rush, the
system is deadlocked: every philosopher holds its left fork and
waits forever for its right fork (circular wait).
Philosopher 0 ate 0 time(s)
Philosopher 1 ate 0 time(s)
Philosopher 2 ate 0 time(s)
Philosopher 3 ate 0 time(s)
```

```
Philosopher 4 ate 0 time(s)
```

This is a genuine deadlock: all 5 philosophers successfully grabbed their LEFT fork, then all 5 blocked forever waiting for their RIGHT fork, which is held by the next philosopher in the circle. No philosopher ever ate (0 meals each) and the program made no further progress, confirming all four deadlock conditions held simultaneously.

## Part B: Deadlock Prevention via Resource Ordering (source: q3b_ordering.c)

Forks are numbered 0-4; every philosopher picks up the LOWER-numbered fork first, then the HIGHER-numbered one.

```
// Q3 Part B: Deadlock prevention using resource (fork) ordering.
// Each philosopher acquires the LOWER-numbered fork first, then the
// HIGHER-numbered fork. This breaks circular wait: philosopher 4 (forks 4,0)
// now grabs fork 0 first, same as philosopher 0, so at least one philosopher
// cannot hold a fork while waiting on another that is itself held-and-wanted
// in a full cycle.

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

#define N 5
#define MEALS_PER_PHILOSOPHER 5   // run a bounded number of meals so it terminates

pthread_mutex_t forks[N];
int meals_eaten[N] = {0};

void msleep(int ms) {
    struct timespec ts = {ms / 1000, (ms % 1000) * 1000000L};
    nanosleep(&ts, NULL);
}

void *philosopher(void *arg) {
    int id = (int)(long)arg;
    int left = id;
    int right = (id + 1) % N;
    int first = left < right ? left : right;   // lower-numbered fork
    int second = left < right ? right : left;  // higher-numbered fork

    while (meals_eaten[id] < MEALS_PER_PHILOSOPHER) {
        printf("Philosopher %d is thinking\n", id);
        msleep(20);

        pthread_mutex_lock(&forks[first]);
        printf("Philosopher %d picked up fork %d\n", id, first);
        pthread_mutex_lock(&forks[second]);
        printf("Philosopher %d picked up fork %d\n", id, second);

        printf("Philosopher %d is EATING (meal %d)\n", id, meals_eaten[id] + 1);
        meals_eaten[id]++;
        msleep(20);

        pthread_mutex_unlock(&forks[second]);
        pthread_mutex_unlock(&forks[first]);
        printf("Philosopher %d released forks %d and %d\n", id, first, second);
    }
```

```
    return NULL;
}

int main() {
    pthread_t th[N];
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < N; i++) pthread_mutex_init(&forks[i], NULL);
    for (long i = 0; i < N; i++) pthread_create(&th[i], NULL, philosopher, (void *)i);
    for (int i = 0; i < N; i++) pthread_join(th[i], NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);
    double elapsed = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("\n===== REPORT (Part B: resource ordering) =====\n");
    printf("No deadlock occurred. All philosophers completed %d meals each.\n",
MEALS_PER_PHILOSOPHER);
    for (int i = 0; i < N; i++)
        printf("Philosopher %d ate %d time(s)\n", i, meals_eaten[i]);
    printf("Total execution time: %.6f seconds\n", elapsed);
    return 0;
}
```

Actual Output (Part B) — excerpt

```
Philosopher 4 picked up fork 0
Philosopher 4 picked up fork 4
Philosopher 4 is EATING (meal 1)
Philosopher 3 picked up fork 3
Philosopher 2 picked up fork 2
Philosopher 1 picked up fork 1
Philosopher 3 picked up fork 4
Philosopher 3 is EATING (meal 1)
Philosopher 4 released forks 0 and 4
...
Philosopher 0 picked up fork 0
Philosopher 0 picked up fork 1
Philosopher 0 is EATING (meal 5)
Philosopher 0 released forks 0 and 1

===== REPORT (Part B: resource ordering) =====
No deadlock occurred. All philosophers completed 5 meals each.
Philosopher 0 ate 5 time(s)
Philosopher 1 ate 5 time(s)
Philosopher 2 ate 5 time(s)
Philosopher 3 ate 5 time(s)
Philosopher 4 ate 5 time(s)
Total execution time: 0.343574 seconds
```

## Part C: Waiter Solution (source: q3c_waiter.c)

A counting semaphore "room", initialized to N-1 = 4, must be acquired before a philosopher may try picking up forks (naive left-then-right order).

```
// Q3 Part C: Waiter (arbitrator) solution.
// A counting semaphore "room" initialized to N-1 (4) ensures at most 4 of
// the 5 philosophers may attempt to pick up forks simultaneously, which
// makes circular wait for all 5 forks impossible (naive left-then-right
// order is fine here).
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define N 5
#define MEALS_PER_PHILOSOPHER 5

pthread_mutex_t forks[N];
sem_t room;   // at most N-1 philosophers may try to eat at once
int meals_eaten[N] = {0};

void msleep(int ms) {
    struct timespec ts = {ms / 1000, (ms % 1000) * 1000000L};
    nanosleep(&ts, NULL);
}

void *philosopher(void *arg) {
    int id = (int)(long)arg;
    int left = id;
    int right = (id + 1) % N;

    while (meals_eaten[id] < MEALS_PER_PHILOSOPHER) {
        printf("Philosopher %d is thinking\n", id);
        msleep(20);

        printf("Philosopher %d requesting permission from waiter\n", id);
        sem_wait(&room);                      // ask the waiter for permission

        pthread_mutex_lock(&forks[left]);
        printf("Philosopher %d picked up left fork %d\n", id, left);
        pthread_mutex_lock(&forks[right]);
        printf("Philosopher %d picked up right fork %d\n", id, right);

        printf("Philosopher %d is EATING (meal %d)\n", id, meals_eaten[id] + 1);
        meals_eaten[id]++;
        msleep(20);

        pthread_mutex_unlock(&forks[right]);
        pthread_mutex_unlock(&forks[left]);
        sem_post(&room);                      // leave the room, free a slot
        printf("Philosopher %d released forks and left the room\n", id);
    }
    return NULL;
}

int main() {
    pthread_t th[N];
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    sem_init(&room, 0, N - 1);   // waiter allows at most 4 philosophers in
    for (int i = 0; i < N; i++) pthread_mutex_init(&forks[i], NULL);
    for (long i = 0; i < N; i++) pthread_create(&th[i], NULL, philosopher, (void *)i);
    for (int i = 0; i < N; i++) pthread_join(th[i], NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);
    double elapsed = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
```

```
    printf("\n===== REPORT (Part C: waiter solution) =====\n");
    printf("No deadlock occurred. All philosophers completed %d meals each.\n",
MEALS_PER_PHILOSOPHER);
    for (int i = 0; i < N; i++)
        printf("Philosopher %d ate %d time(s)\n", i, meals_eaten[i]);
    printf("Total execution time: %.6f seconds\n", elapsed);
    sem_destroy(&room);
    return 0;
}
```

Actual Output (Part C) — excerpt

```
Philosopher 0 is thinking
Philosopher 0 requesting permission from waiter
Philosopher 0 picked up left fork 0
Philosopher 0 picked up right fork 1
Philosopher 0 is EATING (meal 1)
Philosopher 4 requesting permission from waiter
Philosopher 4 picked up left fork 4
Philosopher 3 requesting permission from waiter
Philosopher 3 picked up left fork 3
Philosopher 2 requesting permission from waiter
Philosopher 2 picked up left fork 2
Philosopher 1 requesting permission from waiter
Philosopher 4 picked up right fork 0
Philosopher 4 is EATING (meal 1)
Philosopher 0 released forks and left the room
...
Philosopher 1 released forks and left the room

===== REPORT (Part C: waiter solution) =====
No deadlock occurred. All philosophers completed 5 meals each.
Philosopher 0 ate 5 time(s)
Philosopher 1 ate 5 time(s)
Philosopher 2 ate 5 time(s)
Philosopher 3 ate 5 time(s)
Philosopher 4 ate 5 time(s)
Total execution time: 0.523688 seconds
```

Required Analysis

| Solution | Deadlock? | Meals Eaten (each) | Starvation Observed? | Execution Time |
|---|---|---|---|---|
| A: Naive left-then-right | YES — permanent, watchdog confirmed | 0 (all) | N/A — total deadlock, not starvation | Never terminates (killed by watchdog) |
| B: Resource ordering | No | 5 (all) | No — all 5 finished equally | 0.343574 s |
| C: Waiter (room semaphore) | No | 5 (all) | No — all 5 finished equally | 0.523688 s |

Comparison of the Two Prevention Methods

Resource ordering (Part B) breaks the circular-wait condition directly and needs no extra synchronization primitive beyond the existing fork mutexes, so it has the lowest overhead of the two (0.34s here vs 0.52s

for the waiter). It requires philosophers to know a global numbering, which is easy for a fixed table of 5 but less natural to extend to dynamically-changing resource sets. The waiter solution (Part C) instead limits concurrency at the table level (an extra semaphore acquisition per philosopher, hence the added overhead observed), but it is conceptually simpler to reason about and generalizes well to more complex resource-acquisition orders where a consistent global ordering may not be easy to define.

## Real-World Extension: Banking Threads

If Thread A transfers money from Account X to Account Y (locking X then Y) while Thread B simultaneously transfers from Account Y to Account X (locking Y then X), the two threads can deadlock exactly like the naive philosophers: A holds X's lock and waits for Y, B holds Y's lock and waits for X — a 2-process circular wait. The same fix as Part B applies directly: always lock accounts in a fixed order (e.g. by ascending account ID) regardless of which direction the transfer goes, which guarantees both threads request locks in the same global order and cannot form a cycle.

# Question 4: Sleeping Barber and Service-Queue Synchronization

## Conceptual Questions

**1. What does the barber represent in a computing system?**

The barber represents a single worker/server resource — a CPU core, a server process, or a worker thread — that processes one request/task at a time and is idle (sleeping) when there is no work available.

**2. What do the barber chair and waiting chairs represent?**

The barber chair represents the single active execution slot — the request currently being processed. The waiting chairs represent a bounded queue/buffer of pending requests that are accepted but not yet being serviced.

**3. Why should the barber sleep when there are no customers?**

So the worker thread consumes no CPU cycles while idle. Sleeping (blocking on a semaphore) lets the OS scheduler give the CPU to other useful work instead of busy-waiting/polling for a customer that hasn't arrived yet, which would waste CPU for no benefit.

**4. What is the difference between blocking and busy waiting?**

Blocking means the thread is suspended by the OS (consuming no CPU) until the awaited condition/semaphore is signaled, at which point the scheduler wakes it. Busy waiting means the thread stays runnable and repeatedly polls a condition in a tight loop, consuming CPU cycles the entire time it waits, even though it accomplishes nothing until the condition becomes true.

**5. What happens when the waiting queue is full?**

A newly arriving customer/request that finds no free waiting chair is rejected immediately (turned away) rather than being queued indefinitely — exactly as seen in the experiments below when customers 'left because no chair was available'.

**6. Why must access to the waiting-chair count be protected?**

Multiple customer threads may arrive and check/update the waiting_count and the shared queue simultaneously, while the barber thread also removes entries from the same queue. Without a mutex, two customers could both read the same count as 'not full' and both proceed to occupy what they believe is a free chair, corrupting the count or the queue structure (a race condition), so the mutex is required around every read/update of that shared state.

**7. How is the Sleeping Barber problem related to a web server?**

A web server is directly analogous: the barber is a worker thread/process, incoming HTTP connections are customers, and the fixed-size TCP accept/listen backlog queue is the bounded set of waiting chairs. When the backlog is full, new connection attempts are refused (connection reset) just like a customer leaving when no chair is free, and idle worker threads sleep (blocked on accept()) until a request arrives.

**8. How is it different from the Producer-Consumer problem?**

In Sleeping Barber, if the buffer (waiting chairs) is full, the producer (customer) is immediately turned away/rejected rather than blocking to wait for space. In classic bounded Producer-Consumer (Q2), a producer that finds the buffer full simply BLOCKS on the empty semaphore until a slot frees up — it does not give up. Sleeping Barber therefore models request rejection/dropping, while Producer-Consumer models backpressure/throttling.

## Programming Assignment (source: q4_barber.c)

1 barber, 1 barber chair, a configurable number of waiting chairs, 10 customers arriving at (roughly) fixed intervals per experiment.

```
// Q4: Sleeping Barber problem.
// 1 barber, 1 barber chair, WAITING_CHAIRS waiting chairs, NUM_CUSTOMERS
// customers arriving at random intervals. Configurable via argv so the three
// required experiments can vary waiting-chair count / arrival / service speed.
//
// Usage: ./q4_barber <waiting_chairs> <arrival_ms> <service_ms>

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define NUM_CUSTOMERS 10

int WAITING_CHAIRS = 3;
int arrival_ms = 200;
int service_ms = 200;

int waiting_count = 0;          // customers currently in waiting chairs
int max_queue_len = 0;
int served_count = 0;
int rejected_count = 0;
int barber_sleep_count = 0;

pthread_mutex_t mutex;
sem_t customers;       // number of customers waiting for barber (signals barber)
sem_t barber_ready;    // barber signals it's ready to cut hair
sem_t customer_done[NUM_CUSTOMERS]; // one-per-customer "your haircut is finished" signal
(unused simplification, see note)

// We identify whose turn it is with a simple FIFO ticket queue.
int queue_ids[NUM_CUSTOMERS];
int q_head = 0, q_tail = 0;

int total_processed = 0; // customers who have been served OR rejected

void msleep(int ms) {
    struct timespec ts = {ms / 1000, (ms % 1000) * 1000000L};
    nanosleep(&ts, NULL);
}

void *barber(void *arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        if (total_processed >= NUM_CUSTOMERS && waiting_count == 0) {
            pthread_mutex_unlock(&mutex);
            break; // all customers handled, barber shop closes
        }
        pthread_mutex_unlock(&mutex);

        printf("Barber is sleeping\n");
        barber_sleep_count++;
        sem_wait(&customers);      // sleep until a customer is waiting (or shop closes)
```

```
        pthread_mutex_lock(&mutex);
        if (waiting_count == 0) {
            // Woken only to notice shop is closing with nobody left.
            pthread_mutex_unlock(&mutex);
            continue;
        }
        int cust_id = queue_ids[q_head];
        q_head = (q_head + 1) % NUM_CUSTOMERS;
        waiting_count--;
        pthread_mutex_unlock(&mutex);

        printf("Barber woke up\n");
        printf("Barber is serving Customer %d\n", cust_id);
        sem_post(&barber_ready);   // tell that customer to come sit in the chair

        msleep(service_ms);        // haircut duration
        printf("Barber finished with Customer %d\n", cust_id);

        pthread_mutex_lock(&mutex);
        served_count++;
        total_processed++;
        pthread_mutex_unlock(&mutex);
    }
    printf("Barber shop is closing (all customers handled)\n");
    return NULL;
}

void *customer(void *arg) {
    int id = (int)(long)arg;
    printf("Customer %d arrived\n", id);

    pthread_mutex_lock(&mutex);
    if (waiting_count < WAITING_CHAIRS) {
        queue_ids[q_tail] = id;
        q_tail = (q_tail + 1) % NUM_CUSTOMERS;
        waiting_count++;
        if (waiting_count > max_queue_len) max_queue_len = waiting_count;
        printf("Customer %d is waiting | Occupied chairs: %d\n", id, waiting_count);
        pthread_mutex_unlock(&mutex);

        sem_post(&customers);      // notify barber a customer is waiting
        sem_wait(&barber_ready);   // wait until barber calls this customer
        // Barber is now cutting hair (simulated in barber thread); this
        // thread just waits for the whole service duration to "receive" it.
        msleep(service_ms);
        printf("Customer %d received haircut and leaves\n", id);
    } else {
        rejected_count++;
        total_processed++;   // rejected customers still count as "handled"
        printf("Customer %d left because no chair was available\n", id);
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc >= 4) {
        WAITING_CHAIRS = atoi(argv[1]);
        arrival_ms = atoi(argv[2]);
        service_ms = atoi(argv[3]);
    }
```

```
    pthread_mutex_init(&mutex, NULL);
    sem_init(&customers, 0, 0);
    sem_init(&barber_ready, 0, 0);

    printf("=== Sleeping Barber: waiting_chairs=%d arrival=%dms service=%dms ===\n",
           WAITING_CHAIRS, arrival_ms, service_ms);

    pthread_t barber_thread;
    pthread_create(&barber_thread, NULL, barber, NULL);

    pthread_t cust_threads[NUM_CUSTOMERS];
    for (long i = 0; i < NUM_CUSTOMERS; i++) {
        msleep(arrival_ms);   // random-ish delay before each arrival
        pthread_create(&cust_threads[i], NULL, customer, (void *)(i + 1));
    }

    for (int i = 0; i < NUM_CUSTOMERS; i++) pthread_join(cust_threads[i], NULL);

    // Ensure barber wakes up one final time to notice the shop should close,
    // in case it is sleeping on sem_wait with nothing left to serve.
    sem_post(&customers);

    pthread_join(barber_thread, NULL);

    printf("\n===== SUMMARY =====\n");
    printf("Customers served:   %d\n", served_count);
    printf("Customers rejected: %d\n", rejected_count);
    printf("Times barber slept: %d\n", barber_sleep_count);
    printf("Maximum queue length reached: %d\n", max_queue_len);

    pthread_mutex_destroy(&mutex);
    sem_destroy(&customers);
    sem_destroy(&barber_ready);
    return 0;
}
```

Experiment 1: 3 chairs, slow arrival (300ms), fast service (100ms)

```
=== Sleeping Barber: waiting_chairs=3 arrival=300ms service=100ms ===
Barber is sleeping
Customer 1 arrived
Customer 1 is waiting | Occupied chairs: 1
Barber woke up
Barber is serving Customer 1
Barber finished with Customer 1
Barber is sleeping
Customer 1 received haircut and leaves
...
Customer 10 arrived
Customer 10 is waiting | Occupied chairs: 1
Barber woke up
Barber is serving Customer 10
Barber finished with Customer 10
Barber shop is closing (all customers handled)
Customer 10 received haircut and leaves

===== SUMMARY =====
Customers served:   10
Customers rejected: 0
Times barber slept: 10
Maximum queue length reached: 1
```

With slow arrivals and fast service, the barber comfortably serves every customer immediately; nobody is ever rejected and the queue never exceeds 1.

Experiment 2: 3 chairs, fast arrival (80ms), slow service (400ms)

```
=== Sleeping Barber: waiting_chairs=3 arrival=80ms service=400ms ===
Barber is sleeping
Customer 1 arrived
Customer 1 is waiting | Occupied chairs: 1
Barber woke up
Barber is serving Customer 1
Customer 2 arrived
Customer 2 is waiting | Occupied chairs: 1
Customer 3 arrived
Customer 3 is waiting | Occupied chairs: 2
Customer 4 arrived
Customer 4 is waiting | Occupied chairs: 3
Customer 5 arrived
Customer 5 left because no chair was available
Barber finished with Customer 1
...
Customer 7 arrived
Customer 7 left because no chair was available
Customer 8 arrived
Customer 8 left because no chair was available
Customer 9 arrived
Customer 9 left because no chair was available
Customer 10 arrived
Customer 10 left because no chair was available
...
Barber shop is closing (all customers handled)

===== SUMMARY =====
Customers served:   5
Customers rejected: 5
Times barber slept: 5
Maximum queue length reached: 3
```

With fast arrivals and slow service, the 3-chair queue fills almost immediately and half the customers (5 of 10) are turned away.

Experiment 3: 5 chairs, fast arrival (80ms), slow service (400ms)

```
=== Sleeping Barber: waiting_chairs=5 arrival=80ms service=400ms ===
Barber is sleeping
Customer 1 arrived
Customer 1 is waiting | Occupied chairs: 1
Barber woke up
Barber is serving Customer 1
Customer 2 arrived
Customer 2 is waiting | Occupied chairs: 1
Customer 3 arrived
Customer 3 is waiting | Occupied chairs: 2
Customer 4 arrived
Customer 4 is waiting | Occupied chairs: 3
Customer 5 arrived
Customer 5 is waiting | Occupied chairs: 4
...
Customer 7 arrived
Customer 7 is waiting | Occupied chairs: 5
Customer 8 arrived
Customer 8 left because no chair was available
Customer 9 arrived
Customer 9 left because no chair was available
Customer 10 arrived
```

```
Customer 10 left because no chair was available
...
Barber shop is closing (all customers handled)

===== SUMMARY =====
Customers served:   7
Customers rejected: 3
Times barber slept: 7
Maximum queue length reached: 5
```

Increasing the waiting chairs from 3 to 5 (same arrival/service speeds as Experiment 2) reduced rejections from 5 to 3 — more buffering absorbs more of the burst, but with a persistently faster arrival rate than service rate, rejections cannot be eliminated by buffer size alone.

### Performance Table

| Experiment | Chairs | Arrival | Service | Served | Rejected | Barber Slept | Max Queue |
|---|---|---|---|---|---|---|---|
| 1 | 3 | 300ms (slow) | 100ms (fast) | 10 | 0 | 10 | 1 |
| 2 | 3 | 80ms (fast) | 400ms (slow) | 5 | 5 | 5 | 3 |
| 3 | 5 | 80ms (fast) | 400ms (slow) | 7 | 3 | 7 | 5 |

### Sleeping vs. Busy Waiting

In this implementation the barber calls sem_wait(&customers) when idle, which blocks the thread (consuming 0% CPU) until a customer thread calls sem_post — exactly the 'sleeping' behaviour. Note the summary field 'Times barber slept', which counts exactly how many times the barber genuinely went idle. A busy-waiting alternative would instead have the barber loop calling something like 'while (waiting_count == 0) { }' repeatedly, pegging a CPU core at 100% the entire time it has nothing to do — wasteful and something this design deliberately avoids by using a semaphore.

### Mapping to a Real Web-Server / Cloud-Service System

Barber -> a worker thread/process in a thread-pool web server (e.g. an Nginx/Apache worker or a Node.js/async worker). Waiting chairs -> the server's bounded connection/request backlog queue (e.g. the TCP listen() backlog, or an application-level task queue with a fixed size). Customer arriving and being rejected -> a client whose connection is refused or receives an HTTP 503 'Service Unavailable' because the backlog is already full. Barber sleeping -> an idle worker thread blocked on accept()/a queue-wait primitive, consuming no CPU until a new request arrives. This mapping is why the Sleeping Barber problem is a standard teaching model for how bounded request queues and worker pools behave under load in real servers.

## Additional Challenge: 2 Barbers, 5 Waiting Chairs, 20 Customers (source: q4_barber_multi.c)

The shared waiting queue (waiting_count, the FIFO array, q_head/q_tail) is accessed by BOTH barber threads as well as by every customer thread, so it still requires exactly the same synchronization as the single-barber version: the existing mutex protecting the queue plus the existing customers/barber_ready semaphores are sufficient — no new primitive is needed. Because both barbers pull their next customer from the same mutex-protected FIFO, they can never grab the same waiting customer twice.

```c
// Q4 Additional Challenge: 2 barbers, 5 waiting chairs, 20 customers.
// The waiting queue (waiting_count, queue_ids) is SHARED between both
// barber threads, so it still needs the SAME mutex protection as the
// single-barber case -- no new synchronization primitive is required,
// just correct reuse of the existing mutex + semaphores. Both barbers
// call sem_wait(&customers) and pull the next ticket from the same
// FIFO under the same mutex, so they can never both grab the same
// waiting customer.

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define NUM_BARBERS 2
#define NUM_CUSTOMERS 20
#define WAITING_CHAIRS 5
#define ARRIVAL_MS 60
#define SERVICE_MS 300

int waiting_count = 0;
int max_queue_len = 0;
int served_count = 0;
int rejected_count = 0;
int total_processed = 0;

pthread_mutex_t mutex;
sem_t customers;
sem_t barber_ready;

int queue_ids[NUM_CUSTOMERS];
int q_head = 0, q_tail = 0;

void msleep(int ms) {
    struct timespec ts = {ms / 1000, (ms % 1000) * 1000000L};
    nanosleep(&ts, NULL);
}

void *barber(void *arg) {
    int bid = (int)(long)arg;
    while (1) {
        pthread_mutex_lock(&mutex);
        if (total_processed >= NUM_CUSTOMERS && waiting_count == 0) {
            pthread_mutex_unlock(&mutex);
            break;
        }
        pthread_mutex_unlock(&mutex);

        sem_wait(&customers);

        pthread_mutex_lock(&mutex);
        if (waiting_count == 0) {
            pthread_mutex_unlock(&mutex);
            continue;
        }
        int cust_id = queue_ids[q_head];
        q_head = (q_head + 1) % NUM_CUSTOMERS;
        waiting_count--;
        pthread_mutex_unlock(&mutex);
```

```
        printf("Barber %d is serving Customer %d\n", bid, cust_id);
        sem_post(&barber_ready);
        msleep(SERVICE_MS);
        printf("Barber %d finished with Customer %d\n", bid, cust_id);

        pthread_mutex_lock(&mutex);
        served_count++;
        total_processed++;
        pthread_mutex_unlock(&mutex);
    }
    printf("Barber %d going home (shop closing)\n", bid);
    return NULL;
}

void *customer(void *arg) {
    int id = (int)(long)arg;
    printf("Customer %d arrived\n", id);

    pthread_mutex_lock(&mutex);
    if (waiting_count < WAITING_CHAIRS) {
        queue_ids[q_tail] = id;
        q_tail = (q_tail + 1) % NUM_CUSTOMERS;
        waiting_count++;
        if (waiting_count > max_queue_len) max_queue_len = waiting_count;
        printf("Customer %d is waiting | Occupied chairs: %d\n", id, waiting_count);
        pthread_mutex_unlock(&mutex);

        sem_post(&customers);
        sem_wait(&barber_ready);
        msleep(SERVICE_MS);
        printf("Customer %d received haircut and leaves\n", id);
    } else {
        // mutex is still held from the check above
        rejected_count++;
        total_processed++;
        printf("Customer %d left because no chair was available\n", id);
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main() {
    pthread_mutex_init(&mutex, NULL);
    sem_init(&customers, 0, 0);
    sem_init(&barber_ready, 0, 0);

    printf("=== 2-Barber Sleeping Barber: chairs=%d customers=%d ===\n",
           WAITING_CHAIRS, NUM_CUSTOMERS);

    pthread_t barbers[NUM_BARBERS];
    for (long i = 0; i < NUM_BARBERS; i++)
        pthread_create(&barbers[i], NULL, barber, (void *)(i + 1));

    pthread_t cust_threads[NUM_CUSTOMERS];
    for (long i = 0; i < NUM_CUSTOMERS; i++) {
        msleep(ARRIVAL_MS);
        pthread_create(&cust_threads[i], NULL, customer, (void *)(i + 1));
    }
    for (int i = 0; i < NUM_CUSTOMERS; i++) pthread_join(cust_threads[i], NULL);

    for (int i = 0; i < NUM_BARBERS; i++) sem_post(&customers); // wake both barbers to close
    for (int i = 0; i < NUM_BARBERS; i++) pthread_join(barbers[i], NULL);
```

```
    printf("\n===== SUMMARY (2 barbers) =====\n");
    printf("Customers served:   %d\n", served_count);
    printf("Customers rejected: %d\n", rejected_count);
    printf("Maximum queue length reached: %d\n", max_queue_len);
    return 0;
}
```

Actual Output (Additional Challenge) — excerpt

```
=== 2-Barber Sleeping Barber: chairs=5 customers=20 ===
Customer 1 arrived
Customer 1 is waiting | Occupied chairs: 1
Barber 1 is serving Customer 1
Customer 2 arrived
Customer 2 is waiting | Occupied chairs: 1
Barber 2 is serving Customer 2
...
Customer 9 arrived
Customer 9 is waiting | Occupied chairs: 5
Customer 10 arrived
Customer 10 left because no chair was available
...
Customer 18 arrived
Customer 18 left because no chair was available
Customer 19 arrived
Customer 19 left because no chair was available
Customer 20 arrived
Customer 20 left because no chair was available
...
Barber 1 finished with Customer 17
Barber 1 going home (shop closing)
Barber 2 going home (shop closing)

===== SUMMARY (2 barbers) =====
Customers served:   13
Customers rejected: 7
Maximum queue length reached: 5
```

With 2 barbers instead of 1 (same 5 chairs, same fast-arrival/slow-service ratio as Experiment 3), throughput roughly doubled — 13 of 20 customers served instead of 7 of 10 — confirming that adding a second worker relieves pressure on the bounded queue exactly as adding server capacity does in a real load-balanced service, while the existing mutex/semaphore design required no modification to remain correct.